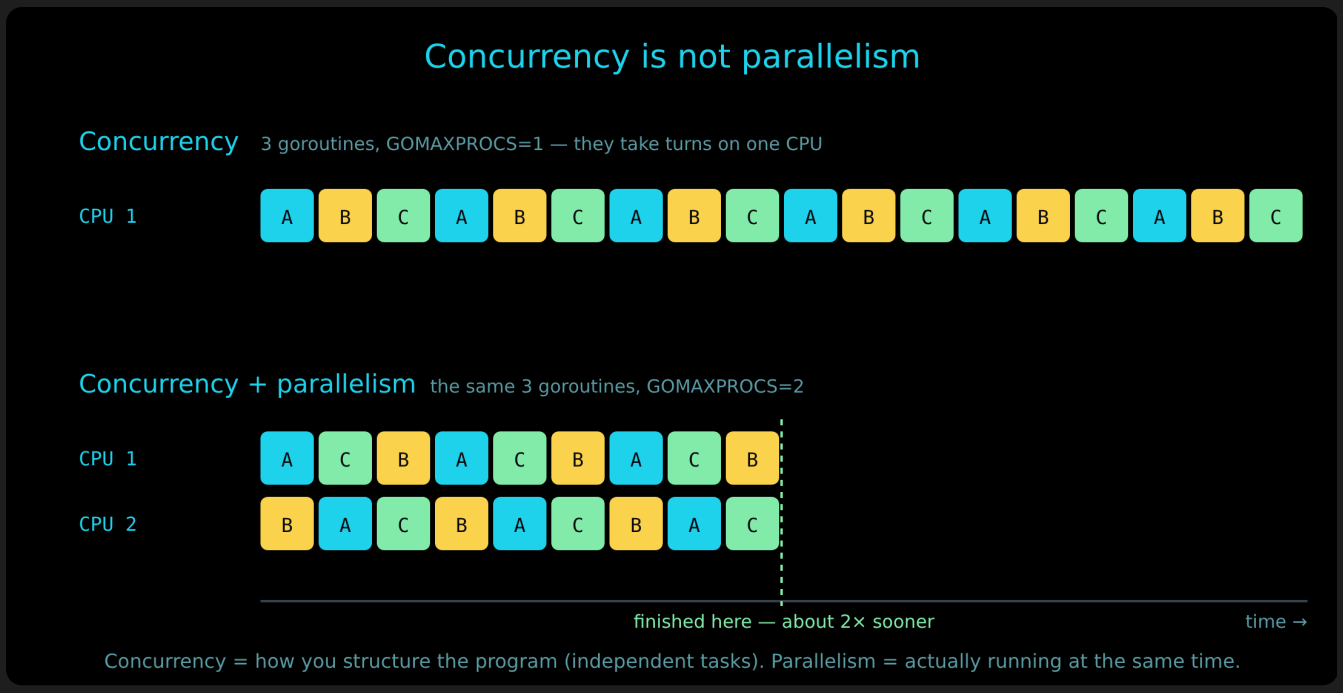
Before start, make sure you've read the following materials:

- Tour of Go: Goroutines, Channels, Buffered Channels
- Go Blog: Concurrency is not parallelism (Rob Pike's talk)
- Go Blog: Container-aware GOMAXPROCS
- Go by Example: Goroutines, WaitGroups
- The Go scheduler
- Go scheduler: not cooperative anymore?
- Concurrency: goroutines and channels (intermediate, optional)
- The GC now finds leaked goroutines (advanced, optional)

> **Note**
>
> All outputs in this tutorial were produced with **Go 1.24** on a machine with **2 CPU cores**. Your numbers will be different — on the defence, explain what **you** see. Channels are used here only as a place where a goroutine waits; the three Tour of Go pages above are enough.

Part 1: Concurrency vs parallelism

> **Important**
>
> ***Concurrency** is about **structure**: a program is made of independent tasks that can make progress without waiting for each other. **Parallelism** is about **execution**: several tasks really run at the same moment on different CPU cores. — "Concurrency is about dealing with lots of things at once. Parallelism is about doing lots of things at once." (Rob Pike)*



Two kinds of work

	CPU-bound	I/O-bound
Spends its time…	calculating	**waiting**: database, network, disk, `time.Sleep`
Example	counting primes, image resizing, hashing	an HTTP request, a SQL query
Gets faster with more goroutines when…	there are more free **CPU cores**	almost always — waiting goroutines don't use the CPU

Let's measure it

```go
// CPU-bound: count primes in [from, to) — deliberately slow
func countPrimes(from, to int) int { /* ... */ }

func cpuWork(workers int) (int, time.Duration) {
    const N = 6_000_000
    start := time.Now()
    results := make([]int, workers)
    var wg sync.WaitGroup
    chunk := N / workers
    for w := 0; w < workers; w++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            results[w] = countPrimes(w*chunk, (w+1)*chunk)
        }()
    }
    wg.Wait()
    // ... add up results
}

// I/O-bound: each "request" just waits 100 ms
func ioWork(workers int) time.Duration {
    start := time.Now()
    var wg sync.WaitGroup
    for w := 0; w < workers; w++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            time.Sleep(100 * time.Millisecond) // like waiting for a database
        }()
    }
    wg.Wait()
    return time.Since(start)
}
```

```
$ ./demo                    # GOMAXPROCS=2 (2 cores)
CPU work, 1 goroutine(s): 412849 primes in 2.648s
CPU work, 2 goroutine(s): 412849 primes in 1.488s
CPU work, 4 goroutine(s): 412849 primes in 1.448s
I/O work,    1 goroutine(s): 101ms
I/O work,   10 goroutine(s): 101ms
I/O work, 1000 goroutine(s): 102ms

$ GOMAXPROCS=1 ./demo       # only one goroutine runs at a time
CPU work, 1 goroutine(s): 412849 primes in 2.739s
CPU work, 2 goroutine(s): 412849 primes in 2.72s
CPU work, 4 goroutine(s): 412849 primes in 2.65s
I/O work,    1 goroutine(s): 100ms
I/O work,   10 goroutine(s): 101ms
I/O work, 1000 goroutine(s): 103ms
```

Question

1. With `GOMAXPROCS=1`, why doesn't the CPU work get faster with more goroutines, but 1000 "requests" still finish in 100 ms?

2. With 2 cores, 4 goroutines are not faster than 2. Why? And why is 2 goroutines not exactly 2× faster than 1? (Hint: are bigger numbers harder to check?)

Task

Task 1: Concurrency vs parallelism

1. Write `cpuWork` and `ioWork` and run them with the default settings and with `GOMAXPROCS=1`. Fill a table with your times. How many cores does your laptop have (`runtime.NumCPU()`)?

2. Try `cpuWork` with as many goroutines as you have cores, and with twice as many. Where does the speed-up stop?

Participation Task: give one real example from a web server for each of: concurrency without parallelism, and concurrency with parallelism.

Part 2: Goroutines

A goroutine is a function that runs concurrently with the rest of the program. You start one by putting `go` in front of a function call:

```
go doSomething(x)            // a named function
go func() { /* ... */ }()    // an anonymous function (Week 6 closures)
```

`main` doesn't wait

```
func say(msg string) {
    for i := 0; i < 3; i++ {
        fmt.Println(msg, i)
    }
}

func main() {
    go say("goroutine")
    say("main")
}
```

```
main 0
main 1
main 2
```

> **Question**
>
> **Where are the `goroutine` lines? Add `time.Sleep(100 * time.Millisecond)` inside the loop (as in the Tour of Go) and run it several times. Why can the output be different every time, and why is `goroutine 2` sometimes missing?**

> **Important**
>
> **When `main` returns, the program exits — all other goroutines are stopped immediately, wherever they are. No `defer` in them runs.**

Waiting for goroutines: `sync.WaitGroup`

```go
func main() {
    var wg sync.WaitGroup
    for i := 1; i <= 5; i++ {
        wg.Add(1)          // 1. count the goroutine BEFORE starting it
        go func() {
            defer wg.Done() // 2. say "I'm finished" when it returns
            fmt.Println("worker", i, "done")
        }()
    }
    wg.Wait()              // 3. block until the counter is 0
    fmt.Println("all workers finished")
}
```

```
worker 5 done
worker 1 done
worker 2 done
worker 3 done
worker 4 done
all workers finished
```

> **Note**
>
> **Since Go 1.22 every loop iteration has its own `i` (Week 6), so each goroutine prints its own number. Since Go 1.25 you can write `wg.Go(func() { ... })` — it does `Add(1)`, `go` and `Done()` for you.**

> ⚠ **Caution**
>
> **Two classic mistakes**

```
var wg sync.WaitGroup
counter := 0
for i := 0; i < 1000; i++ {
    go func() {
        wg.Add(1)       // WRONG: Wait() may run before this line
        defer wg.Done()
        counter++       // WRONG: 1000 goroutines change one variable
    }()
}
wg.Wait()
fmt.Println("counter:", counter)
```

```
counter: 889
counter: 928
counter: 942

$ go run -race .
WARNING: DATA RACE
```

`counter++` is three steps: read, add, write. Two goroutines can read the same value and both write `n+1`. This is a data race. Always test concurrent code with the race detector: `go run -race`, `go test -race`.

A panic in a goroutine kills the whole program

```
func main() {
    defer func() { fmt.Println("main recovered:", recover()) }()
    go func() {
        var m map[string]int
        m["x"] = 1 // panic inside a goroutine
    }()
    time.Sleep(100 * time.Millisecond)
}
```

```
panic: assignment to entry in nil map

goroutine 18 [running]:
main.main.func2()
```

`recover` in `main` does nothing: it only catches panics of its own goroutine (Week 6). If a goroutine may panic, it needs its own deferred `recover`.

Goroutines are cheap — but not free

```
const N = 100_000
stop := make(chan struct{})
for i := 0; i < N; i++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        <-stop // wait until main closes the channel
    }()
}
```

```
goroutines alive: 100001
stack memory: 195 MB (2.0 KB per goroutine)
after close: 1 goroutine(s)
```

	Goroutine	OS thread
Starting stack	2 KB, grows when needed (Week 7)	about 1–8 MB, fixed
Created and scheduled by	the Go runtime	the operating system
Switching between them	cheap, in user space	expensive, through the kernel
How many is "a lot"	hundreds of thousands	thousands

```
BenchmarkGoroutineStartStop-2   2658782   509.7 ns/op   16 B/op   1 allocs/op
BenchmarkFuncCall-2            57793551    19.67 ns/op    0 B/op   0 allocs/op
```

Task

Task 2: First goroutines

1. Fix the `say` program with a `sync.WaitGroup` so that both loops always finish.

2. Run the "two classic mistakes" program 3 times and with `-race`. Fix it twice: with `atomic.Int64` (`sync/atomic`), and by letting each goroutine write to its own element of a `[]int`.

3. Make a goroutine panic, then add a `recover` that keeps the program alive. Where must the `defer` be?

Participation Task: in the WaitGroup example, `worker 5` is printed first almost every time. Run it 10 times, then 10 times with `GOMAXPROCS=1`. Come back to this after Part 3 and explain it.

Task

Task 3: How much does a goroutine cost?

1. Start 100 000 goroutines that wait on a channel. Print `runtime.NumGoroutine()` and the stack memory from `runtime.MemStats` (`StackSys` before and after). How many KB per goroutine?

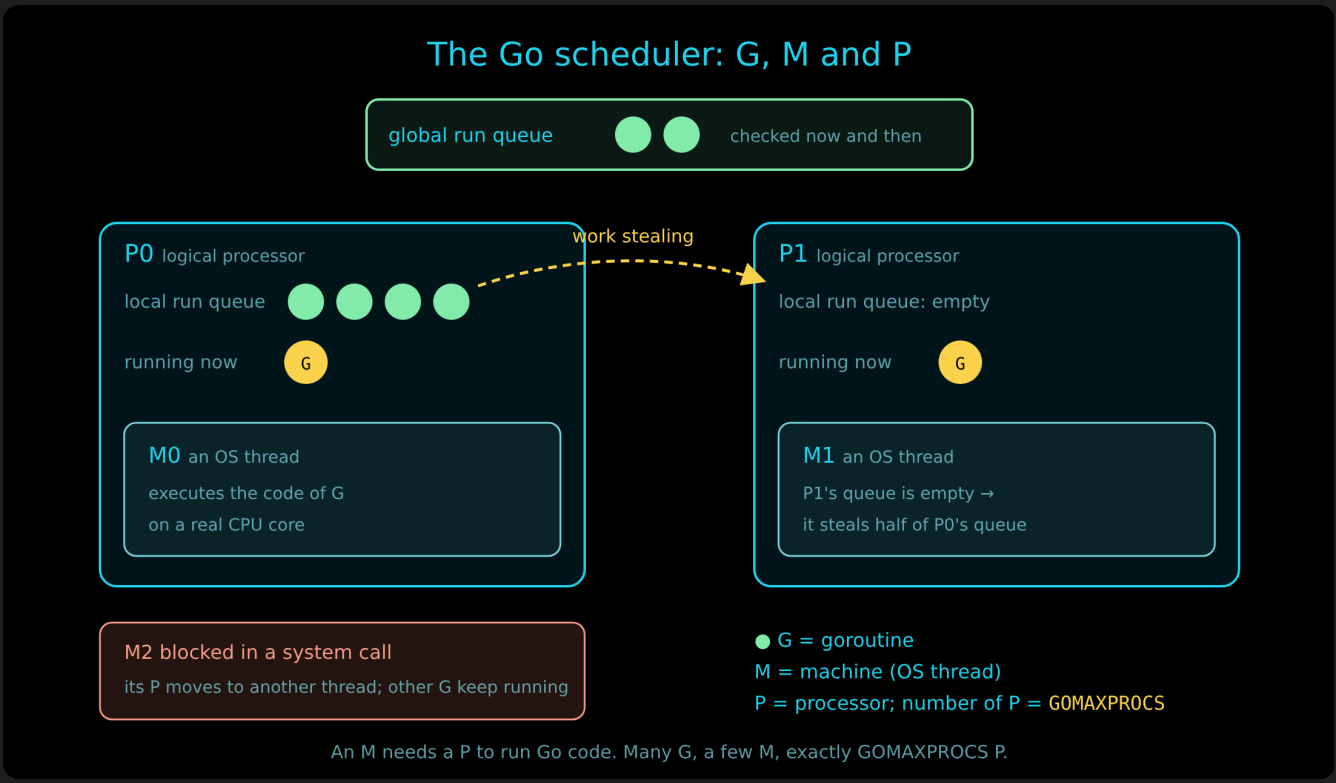
2. How long does it take to start them? Try 1 000 000 if your laptop has enough memory.

3. Write `BenchmarkGoroutineStartStop` (start a goroutine and wait for it) and `BenchmarkFuncCall` (call the same function directly). How many times more expensive is a goroutine? When is it **not** worth starting one?

Part 3: The Go scheduler

The operating system schedules threads. Go puts many goroutines on top of a few threads and decides itself which goroutine runs where. This part of the runtime is the scheduler. It is built from three things:

Letter	Name	What it is
G	goroutine	your function + its stack + its state
M	machine	a real OS thread that executes code
P	processor	a "permission to run Go code" with its own queue of runnable goroutines. There are exactly `GOMAXPROCS` of them



How it works (simplified)

- `go f()` puts the new G into the **local run queue** of the current P.
- Each M with a P takes the next G from its queue and runs it.
- A P with an empty queue first checks the **global run queue**, then **steals** half of another P's queue (**work stealing**). This keeps all cores busy.
- When a G blocks on a channel, a mutex, `time.Sleep` or the network, it is **parked** — the M simply picks another G. No thread is wasted.
- When a G makes a **blocking system call**, its whole M is stuck in the kernel. The P is **handed off** to another M so other goroutines keep running.

`GOMAXPROCS`

```
fmt.Println(runtime.NumCPU())       // cores the program may use
fmt.Println(runtime.GOMAXPROCS(0))  // current value (0 = only read it)
runtime.GOMAXPROCS(1)               // change it (rarely a good idea)
```

By default `GOMAXPROCS` = number of cores. Since Go 1.25 it also respects a container's CPU limit (Docker, Kubernetes) and updates when the limit changes.

Watch the scheduler: `GODEBUG=schedtrace`

```
// 8 goroutines, each busy for 2 seconds, on 2 cores
for i := 0; i < 8; i++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        busy(2 * time.Second)
    }()
}
```

```
$ GODEBUG=schedtrace=500 ./sched
SCHED 0ms: gomaxprocs=2 idleprocs=1 threads=4 spinningthreads=0 needspinning=0 idlethreads=2
runqueue=0 [0 0]
SCHED 504ms: gomaxprocs=2 idleprocs=0 threads=4 spinningthreads=0 needspinning=1
idlethreads=1 runqueue=3 [3 0]
SCHED 1010ms: gomaxprocs=2 idleprocs=0 threads=4 spinningthreads=0 needspinning=1
idlethreads=1 runqueue=1 [2 3]
```

Field	Meaning
`gomaxprocs=2`	number of P
`idleprocs=0`	P with nothing to do — here both are busy
`threads=4`	OS threads (M) that exist
`runqueue=3`	goroutines in the **global** queue
`[3 0]`	goroutines in the **local** queue of P0 and P1

Preemption: nobody can hog a CPU forever

```
func main() {
    runtime.GOMAXPROCS(1) // only one goroutine can run at a time
    go func() {
        for { // a busy loop with no function calls
        }
    }()
    time.Sleep(10 * time.Millisecond)
    fmt.Println("main is still alive")
}
```

```
$ go run .
main is still alive

$ GODEBUG=asyncpreemptoff=1 go run .
(hangs forever)
```

Since Go 1.14 the runtime preempts a goroutine that runs too long (about 10 ms): it sends the thread a signal and switches to another goroutine. Before 1.14 this program hung — that is what the Habr article "not cooperative anymore?" is about.

Blocking system calls create threads

```
run("100 x time.Sleep", 100, func() { time.Sleep(200 * time.Millisecond) })
run("100 x blocking syscall", 100, func() {
    ts := syscall.NsecToTimespec(int64(200 * time.Millisecond))
    syscall.Nanosleep(&ts, nil) // the whole OS thread is blocked
})
```

```
start                       threads created so far: 4
100 x time.Sleep            threads created so far: 4
100 x blocking syscall      threads created so far: 102
```

Question

Both versions wait 200 ms. Why does `time.Sleep` need no new threads, while the system call needs almost one thread per goroutine? Use the words G, M and P.

Task

Task 4: Explore the scheduler

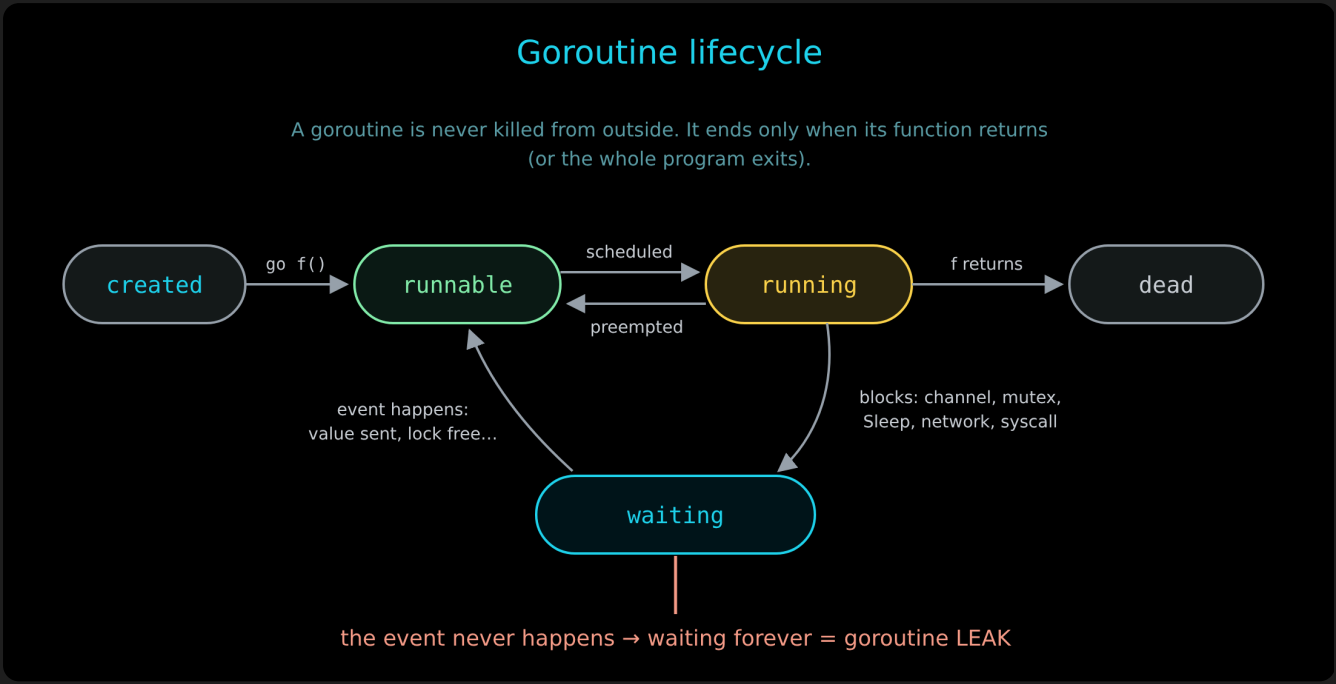
1. Run the 8-busy-goroutines program with `GODEBUG=schedtrace=500` and explain one line, field by field. Then run it with `GOMAXPROCS=1`. What changes?

2. Run the preemption example normally and with `GODEBUG=asyncpreemptoff=1`. Explain the difference.

3. Count threads with `pprof.Lookup("threadcreate").Count()` for `time.Sleep` vs `syscall.Nanosleep`.

4. Now explain the "`worker 5` is printed first" puzzle from Task 2. (Hint: each P has a special slot called `runnext` for the most recently created goroutine.)

Part 4: Goroutine lifecycle and leaks



> **Important**
>
> ***Go has no way to kill a goroutine from outside. A goroutine ends only when its function returns. If it waits for something that never happens, it waits forever: its stack and everything it references stay in memory. This is a goroutine leak.***

Leak 1: nobody takes the result

```
func slowQuery() int {
    time.Sleep(50 * time.Millisecond) // pretend to ask a database
    return 42
}

func fetchLeaky(timeout time.Duration) (int, error) {
    ch := make(chan int) // unbuffered
    go func() {
        ch <- slowQuery() // blocks forever if fetchLeaky already returned
    }()
    select {
    case v := <-ch:
        return v, nil
    case <-time.After(timeout):
        return 0, errors.New("timeout")
    }
}
```

After a timeout `fetchLeaky` returns, nobody will ever read from `ch`, and the goroutine waits on `ch <-` forever. Fix: `make(chan int, 1)` — with a buffer of one, the send always succeeds and the goroutine can finish.

Leak 2: a worker nobody can stop

```
func startTickerLeaky() {
    go func() {
        for {
            time.Sleep(10 * time.Millisecond) // "check something"
        }
    }()
}

// FIX: give the caller a way to say "stop"
func startTicker(ctx context.Context) {
    go func() {
        for {
            select {
            case <-ctx.Done():
                return
            case <-time.After(10 * time.Millisecond):
            }
        }
    }()
}
```

Count them: `runtime.NumGoroutine()`

```
start: 1
after 100 fetchLeaky: 101
after 100 fetchFixed: 101
after 10 leaky tickers: 111
after 10 stoppable tickers: 121
after cancel(): 111
```

Find them: the goroutine profile

```
pprof.Lookup("goroutine").WriteTo(os.Stdout, 1) // import "runtime/pprof"
```

```
goroutine profile: total 111
100 @ 0x46de6e 0x40ba05 0x40b657 0x4d38bd 0x474ac1
#   0x4d38bc    main.fetchLeaky.func1+0x3c  /home/claude/w8/leak/main.go:23

10 @ 0x46de6e 0x471145 0x4d41d8 0x474ac1
#   0x471144    time.Sleep+0x164            /usr/local/go/src/runtime/time.go:338
#   0x4d41d7    main.startTickerLeaky.func1+0x17    /home/claude/w8/leak/main.go:51
```

"100 goroutines are stuck at line 23" — that is the leak. With `WriteTo(os.Stdout, 2)` you also see why each goroutine waits:

```
goroutine 6 [chan send]:
goroutine 7 [chan receive]:
goroutine 8 [select (no cases)]:
```

> **Tip**
>
> **In a real server, import `_ "net/http/pprof"` and open `/debug/pprof/goroutine?debug=1`. In tests, the library `go.uber.org/goleak` fails a test that leaves goroutines behind. Go 1.26 added an experimental goroutine leak profile, and in Go 1.27 it is generally available (`/debug/pprof/goroutineleak`): the garbage collector finds goroutines blocked on channels or locks that nothing can ever wake up.**

The runtime only notices when everything is stuck

```go
func main() {
    ch := make(chan int)
    ch <- 1 // nobody receives
}
```

```
fatal error: all goroutines are asleep - deadlock!
```

If even one goroutine is still running (a server always has some), the runtime cannot tell — leaked goroutines are silent.

Why leaks hurt

```go
func handle(timeout time.Duration) error {
    ch := make(chan *Report)
    go func() { ch <- buildReport() }() // builds a 64 KB report in 20 ms
    select {
    case r := <-ch:
        _ = r
        return nil
    case <-time.After(timeout):
        return errors.New("timeout")
    }
}
```

```
start            goroutines=1     heap=0 MB
after round 1    goroutines=1001  heap=63 MB
after round 2    goroutines=2001  heap=127 MB
after round 3    goroutines=3001  heap=190 MB
```

Each leaked goroutine keeps its 64 KB report alive — the garbage collector can't free it, because the goroutine's stack still points to it (Week 7: reachability). A server that leaks on every timeout runs out of memory after some hours or days.

Tip

Rules that prevent leaks

1. Before writing `go`, answer: **how and when will this goroutine stop?**
2. If a goroutine sends a result that may be ignored, give the channel a **buffer** large enough for all sends.
3. Long-running goroutines take a `context.Context` (or a `done` channel) and return on `ctx.Done()`.
4. Check `runtime.NumGoroutine()` in tests and look at the goroutine profile in production.

Task

Task 5: Find and fix the leaks

1. Write `fetchLeaky` and `startTickerLeaky`. Call them in a loop and print `runtime.NumGoroutine()` after each step.
2. Print the goroutine profile with `debug=1` and `debug=2`. Find the exact line where the leaked goroutines are stuck, and their state.
3. Fix both functions. Prove the fix with the same counters.
4. Write a test that fails if `fetchLeaky` leaves goroutines behind: compare `runtime.NumGoroutine()` before and after (give goroutines a short time to finish).

Task

Task 6: A leaking "server"

1. Write `handle` and `buildReport` as above. Start 1000 `handle` calls with a 5 ms timeout, three rounds, and print goroutines and heap after each round (call `runtime.GC()` first).
2. Fix it in two steps: (a) a buffered channel; (b) pass a `context.WithTimeout` to `buildReport` so it also **stops working** when nobody needs the result.
3. Show that goroutines and heap stay flat after the fix. Why is step (b) better than step (a) alone?

Task

Task 7: Benchmark — how many goroutines are enough?

Count the primes below 200 000 with `CountSeq(n)` (no goroutines) and `CountPar(n, parts)` (split the range into `parts` chunks, one goroutine per chunk).

```
func BenchmarkCountPar(b *testing.B) {
    for _, parts := range []int{2, 8, 1000, 200_000} {
        b.Run(fmt.Sprintf("parts=%d", parts), func(b *testing.B) {
            for i := 0; i < b.N; i++ {
                sink = CountPar(N, parts)
            }
        })
    }
}
```

Requirements:

- First write a test that checks `CountPar` gives the same answer as `CountSeq` for every `parts`.
- Run `go test -bench . -benchmem -cpu 1,2` (or `-cpu 1,4,8` on your laptop). Fill a table.
- Explain: why does `parts=2` on 2 cores not give a 2× speed-up, but `parts=8` comes closer? Why is `parts=200000` the **slowest** of all — even slower than no goroutines?

⭐ Bonus question (for those who want more)

```
runtime.GOMAXPROCS(1)
var wg sync.WaitGroup
for i := 1; i <= 5; i++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        fmt.Print(i, " ")
    }()
}
wg.Wait()
```

What does it print, and is it always the same? The Go specification promises **nothing** about the order of goroutines — so why is it a bug to rely on this order, even if it never changes on your laptop?

Theoretical questions for the defence (in-person with Bakytzhan-agai):

- What is the difference between concurrency and parallelism? Give an example of each.
- What happens to running goroutines when `main` returns?
- Why must `wg.Add(1)` be called before `go`, not inside the goroutine?
- What is a data race? How do you find one?
- Why are goroutines cheaper than OS threads?
- What are G, M and P? What does `GOMAXPROCS` control?
- What is work stealing? What happens when a goroutine makes a blocking system call?
- What is preemption, and what changed in Go 1.14?
- What is a goroutine leak? Name two causes and two ways to find one.

Key takeaways:

- Concurrency is structure, parallelism is execution. Concurrency helps I/O-bound work even on one core; CPU-bound work needs more cores.
- `go f()` starts a goroutine. `main` doesn't wait — use `sync.WaitGroup`, and call `Add` before `go`.
- Goroutines start with a 2 KB stack and are cheap, but not free: don't start one per tiny piece of work.
- The scheduler runs many G on a few M; there are `GOMAXPROCS` P. Idle P steal work; blocked G don't block a thread.
- Goroutines are preempted, so a busy loop can't freeze the program.
- A goroutine can't be killed from outside. Every goroutine needs a way to finish: a buffered channel, a `context`, a `done` channel.
- Find leaks with `runtime.NumGoroutine()` and the goroutine profile; always run concurrent code with `-race`.

Defence will be conducted in the next practice lesson, according to your schedule. In-person with your laptops.
